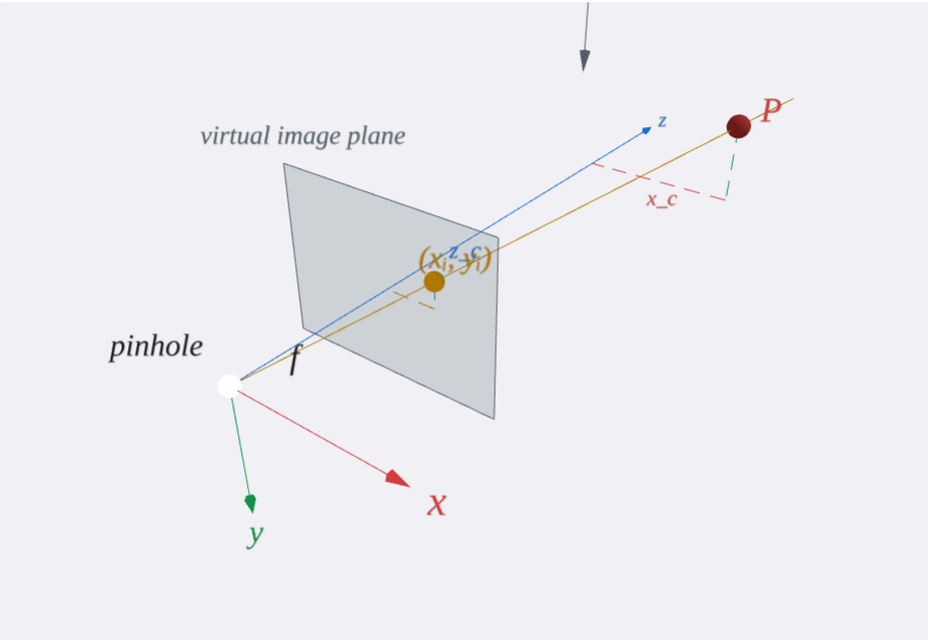
Lecture 2
Camera calibration, PnP, RANSAC & stereo

Robot Control · University of Warsaw

The content layout is greatly inspired by S. Nayar's First Principles of Computer Vision.

Last time: the pinhole camera



Similar triangles: $\frac{x_i}{f} = \frac{x_c}{z_c}, \quad \frac{y_i}{f} = \frac{y_c}{z_c}$

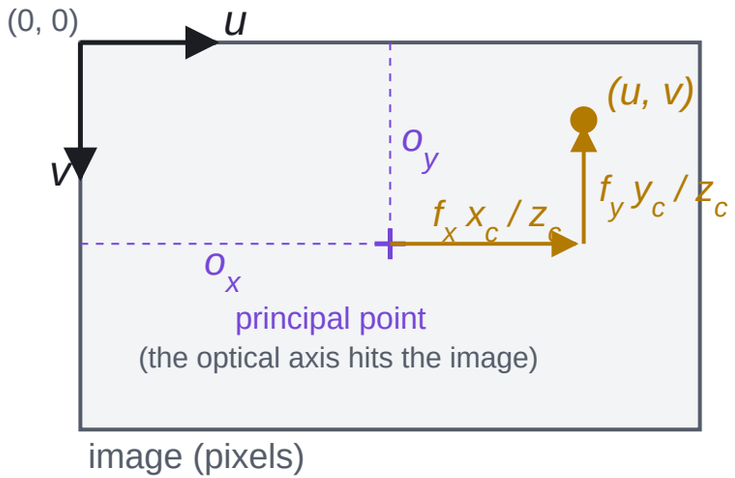


image (pixels)

$$u = f_x \frac{x_c}{z_c} + o_x, \qquad v = f_y \frac{y_c}{z_c} + o_y$$

$$z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

f_x, f_y: focal length in pixels (zoom) · o_x, o_y: principal point (shift)

Last time → today

Lecture 1

- a 3D point **in the camera frame** → pixel: $z_c\,(u, v, 1)^\top = K\,x_c$
- a pixel → a ray $x_c = z_c\,K^{-1}(u, v, 1)^\top$, $z_c > 0$: depth is lost
- rigid motion $x \mapsto Rx + t$

Today

- the camera is **not** at the origin: extrinsics, $P = K[R\,|\,t]$
- how to find K: **calibration**
- where is a known object: **PnP**
- wrong measurements: **RANSAC**
- depth from two cameras: **stereo**

Self-study (not repeated today): lens distortion — lecture 1, part 6. Calibration estimates the distortion coefficients together with K.

Further reading: Szeliski, Computer Vision: Algorithms and Applications, ch. 11–12; Hartley & Zisserman, Multiple View Geometry, ch. 4, 6–7; First Principles of Computer Vision (S. Nayar).

Part 1

The full camera model

Forward imaging model: 3D → 2D

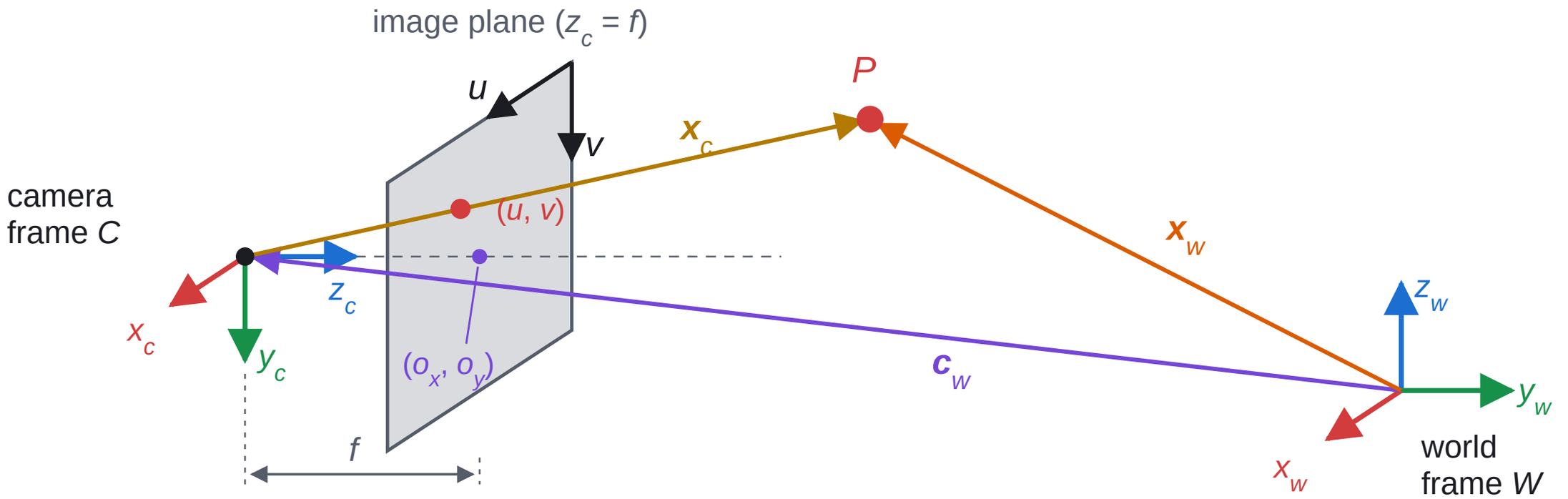


image coordinates

$$\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix}$$

M_{int}

perspective projection
(intrinsics, K)

camera coordinates

$$\mathbf{x}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

M_{ext}

rigid transformation
(extrinsics, R, t)

world coordinates

$$\mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$$

figure after S. Nayar, First Principles of Computer Vision

The projection matrix P

Camera to pixel

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{u}} = M_{int}\,\tilde{\mathbf{x}}_c, \qquad M_{int} = [K \,|\, 0]$$

World to camera

$$\begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{x}}_c = M_{ext}\,\tilde{\mathbf{x}}_w, \qquad \mathbf{x}_c = R(\mathbf{x}_w - \mathbf{c}_w) \;\Rightarrow\; t = -R\,\mathbf{c}_w$$

Combining the two equations gives the full projection matrix P:

$$\tilde{\mathbf{u}} = M_{int} M_{ext}\,\tilde{\mathbf{x}}_w = K[R \,|\, t]\,\tilde{\mathbf{x}}_w = P\,\tilde{\mathbf{x}}_w$$

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}, \quad (u, v) = \left(\frac{\tilde{u}}{\tilde{w}}, \frac{\tilde{v}}{\tilde{w}} \right)$$

after S. Nayar, First Principles of Computer Vision

The projection matrix P: properties

- $P = K[R \mid t]$ is 3×4
- P and kP ($k \neq 0$) give the same pixels: defined **up to scale**
- $\tilde{w}$ = depth z_c of the point (when P is normalised so that $\|(p_{31}, p_{32}, p_{33})\| = 1$)

The camera centre $\mathbf{c}_w$

The pinhole (the centre of projection, where all rays meet), given in world coordinates. In the camera frame it is the origin: $\mathbf{x}_c = R\,\mathbf{c}_w + t = R\,\mathbf{c}_w - R\,\mathbf{c}_w = \mathbf{0}$, since $t = -R\,\mathbf{c}_w$. So

$$P \begin{bmatrix} \mathbf{c}_w \\ 1 \end{bmatrix} = K\,(R\,\mathbf{c}_w + t) = K\,\mathbf{0} = \mathbf{0}$$

$(0, 0, 0)$ is not a homogeneous pixel: the centre is the only point in space without an image. Conversely, from a calibrated P the centre is its null space: $\mathbf{c}_w = -R^\top t$.

Counting degrees of freedom

P: 12 entries, up to scale → **11 DoF**.

K with f_x, f_y, o_x, o_y: 4 + R: 3 + t: 3 = **10**.

> **Question** Which parameter is missing?

The **skew** s in $K = \begin{bmatrix} f_x & s & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$: pixel axes not perpendicular.

Every P whose left 3×3 block is invertible splits uniquely into $K[R \mid t]$ with this K (positive diagonal): $4 + 1 + 3 + 3 = 11$.

Skew in practice

$s \neq 0$ means the pixel axes are not perpendicular. Not the case for digital sensors (OpenCV fixes $s = 0$); it appears when an image is warped with a shear, or for film scanned with a slightly slanted scan direction. We keep s so that every P splits into $K[R \mid t]$.

Part 2

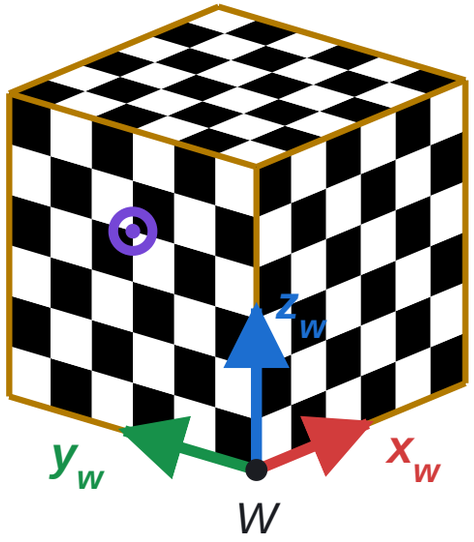
Camera calibration

How to find the projection matrix?

Calibration procedure: steps 1–2

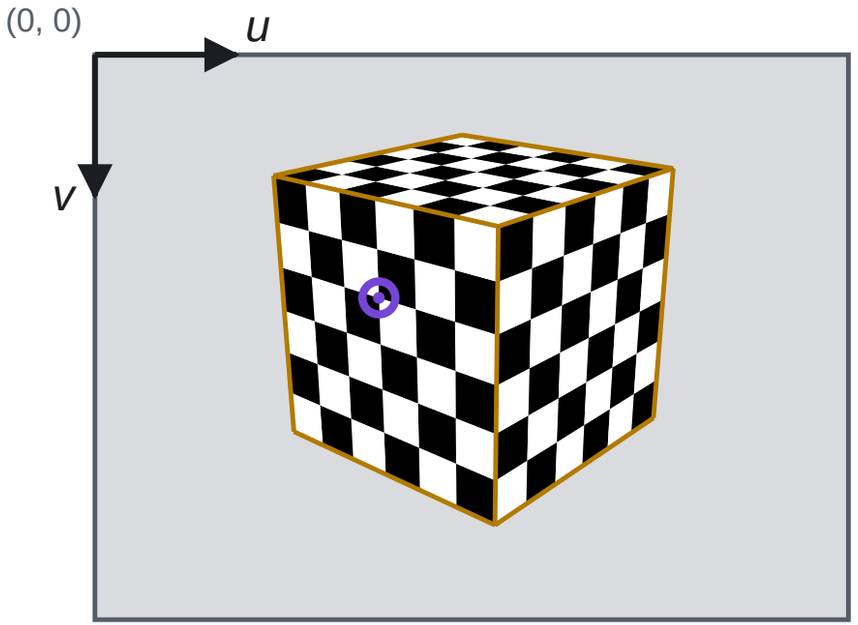
Step 1: capture an image of an object with **known geometry**.

Step 2: identify **correspondences** between 3D scene points and image points.



object of known geometry (squares of 1 cm)

- $\mathbf{x}_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 4 \end{bmatrix}$ (cm)



captured image (640 × 480 pixels)

- $\mathbf{u} = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 241 \\ 207 \end{bmatrix}$ (pixels)

Calibration procedure: step 3

For each corresponding point i, with $\mathbf{x}_w^{(i)} = (x_i, y_i, z_i)$ and pixel (u_i, v_i):

$$\underbrace{\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}}_{\text{known}} \equiv \underbrace{\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}}_{\text{unknown}} \underbrace{\begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}}_{\text{known}} \qquad (\equiv \text{ equal up to scale})$$

Expanding the matrix as equations:

$$u_i = \frac{p_{11}x_i + p_{12}y_i + p_{13}z_i + p_{14}}{p_{31}x_i + p_{32}y_i + p_{33}z_i + p_{34}} \qquad v_i = \frac{p_{21}x_i + p_{22}y_i + p_{23}z_i + p_{24}}{p_{31}x_i + p_{32}y_i + p_{33}z_i + p_{34}}$$

Multiply out the denominators: **linear** in the unknowns p_{jk}

$$p_{11}x_i + p_{12}y_i + p_{13}z_i + p_{14} - u_i x_i\, p_{31} - u_i y_i\, p_{32} - u_i z_i\, p_{33} - u_i\, p_{34} = 0$$

$$p_{21}x_i + p_{22}y_i + p_{23}z_i + p_{24} - v_i x_i\, p_{31} - v_i y_i\, p_{32} - v_i z_i\, p_{33} - v_i\, p_{34} = 0$$

Calibration procedure: step 4

Rearranging the terms: stack the two equations of every point.

$$\underbrace{\begin{bmatrix} x_1 & y_1 & z_1 & 1 & 0 & 0 & 0 & 0 & -u_1x_1 & -u_1y_1 & -u_1z_1 & -u_1 \\ 0 & 0 & 0 & 0 & x_1 & y_1 & z_1 & 1 & -v_1x_1 & -v_1y_1 & -v_1z_1 & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_i & y_i & z_i & 1 & 0 & 0 & 0 & 0 & -u_ix_i & -u_iy_i & -u_iz_i & -u_i \\ 0 & 0 & 0 & 0 & x_i & y_i & z_i & 1 & -v_ix_i & -v_iy_i & -v_iz_i & -v_i \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & z_n & 1 & 0 & 0 & 0 & 0 & -u_nx_n & -u_ny_n & -u_nz_n & -u_n \\ 0 & 0 & 0 & 0 & x_n & y_n & z_n & 1 & -v_nx_n & -v_ny_n & -v_nz_n & -v_n \end{bmatrix}}_{A\ (2n\times 12)\text{: known}} \underbrace{\begin{bmatrix} p_{11} \\ p_{12} \\ p_{13} \\ p_{14} \\ p_{21} \\ p_{22} \\ p_{23} \\ p_{24} \\ p_{31} \\ p_{32} \\ p_{33} \\ p_{34} \end{bmatrix}}_{\mathbf{p}\text{: unknown}} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

$A\mathbf{p} = \mathbf{0}$. 12 entries, but P is defined up to scale (next slide): 11 unknowns, 2 equations per point → at least **6 points**, not all on one plane.

Scale of the projection matrix

The projection matrix acts on homogeneous coordinates. We know that

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \equiv k \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} \qquad (k \neq 0 \text{ is any constant})$$

That is,

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} \equiv k \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

> Projection matrices P and kP give the same pixels. So if $\mathbf{p}$ solves $A\mathbf{p} = \mathbf{0}$, so does $k\mathbf{p}$ — and $\mathbf{p} = \mathbf{0}$ is always a (useless) solution. We must **fix the scale**.

Least squares solution for P

Option 1: set the scale so that $p_{34} = 1$

Option 2: set the scale so that $\|\mathbf{p}\|^2 = 1$

With noise, $A\mathbf{p} = \mathbf{0}$ has no exact solution. We want $A\mathbf{p}$ as close to $\mathbf{0}$ as possible and $\|\mathbf{p}\|^2 = 1$:

$$\min_{\mathbf{p}} \|A\mathbf{p}\|^2 \quad \text{such that} \quad \|\mathbf{p}\|^2 = 1$$

$$\min_{\mathbf{p}} \mathbf{p}^\top A^\top A\, \mathbf{p} \quad \text{such that} \quad \mathbf{p}^\top \mathbf{p} = 1$$

A minimisation with an equality constraint: the method of Lagrange multipliers (next slides).

Why not $p_{34} = 1$?

It also fixes the scale, and then plain least squares works. But $p_{34} = (Kt)_3 = t_z$: if the world origin happens to be close to the camera's $z_c = 0$ plane, $p_{34} \approx 0$ and fixing it to 1 blows up the other entries. The constraint $\|\mathbf{p}\| = 1$ has no such special case.

The Lagrangian method: intuition

Problem

$$\begin{aligned} \text{maximise} \quad & f(x,y) = x^2 y \\ \text{subject to} \quad & g(x,y) = x^2 + y^2 - 1 = 0 \end{aligned}$$

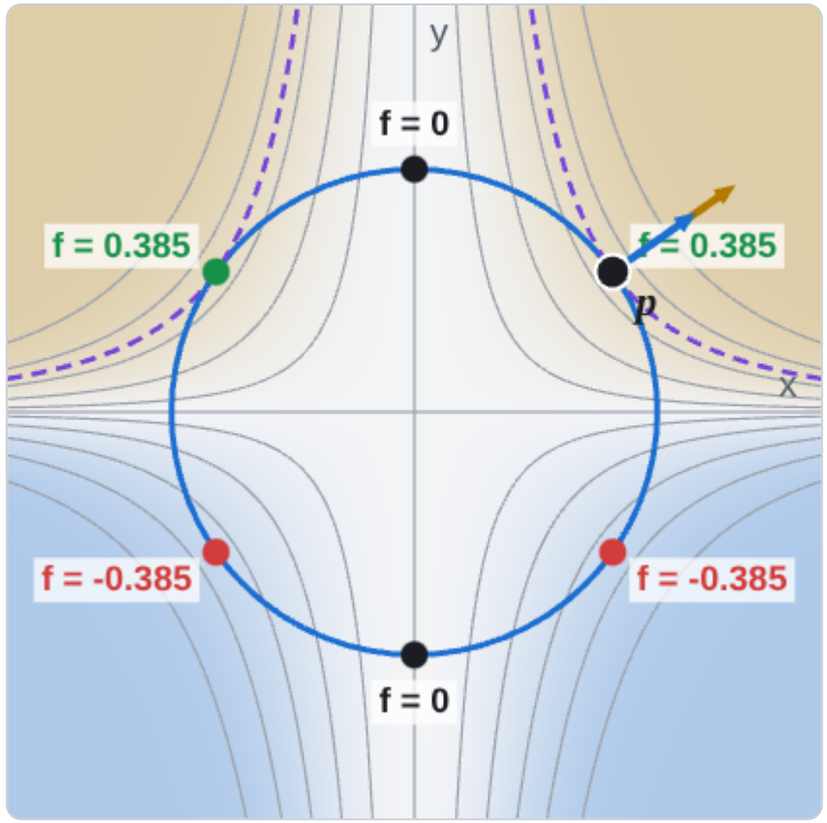
the constraint $g = 0$ is the unit circle

At the optimum the contour of f **touches** the circle — if it crossed it, we could move along the circle to a higher contour. Touching = parallel normals:

$$\nabla f = \lambda \nabla g$$

Solutions: $(0, \pm 1)$, $(\pm\sqrt{2/3}, \pm 1/\sqrt{3})$.

Only **candidates**: evaluate f, pick the best.



- point p on the circle $g = 0$
- the contour line of f through p
- other contour lines of f

$p = (0.82,\ 0.58)$

contour: $f = f(p) = 0.385$

angle$(\nabla f,\ \nabla g) = 0°$

$\nabla f \parallel \nabla g$: the contour touches the circle — a **candidate**. All candidates marked; the best one is found by evaluating f.

From the intuition to the Lagrangian

Optimise $f(x)$ subject to $g(x) = 0$. At the optimum, two conditions:

1. the contour of f touches the constraint: $\nabla f(x) = \lambda \nabla g(x)$
2. we are on the constraint: $g(x) = 0$

Pack both into one function of x **and** λ, the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda\, g(x)$$

Its gradient is zero exactly when both conditions hold:

$$\nabla_x \mathcal{L} = \nabla f(x) - \lambda \nabla g(x) = 0 \quad (1) \text{ touching}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -g(x) = 0 \quad (2) \text{ on the constraint}$$

Solutions are **candidates** (max, min or neither): evaluate f, pick the best.

Example (previous slide)

$$\mathcal{L} = x^2 y - \lambda\,(x^2 + y^2 - 1)$$

$$\partial_x : \; 2xy - 2\lambda x = 0$$

$$\partial_y : \; x^2 - 2\lambda y = 0$$

$$\partial_\lambda : \; x^2 + y^2 = 1$$

→ the six candidates $(0, \pm 1)$, $(\pm\sqrt{2/3}, \pm 1/\sqrt{3})$

Our problem

$f(\mathbf{p}) = \mathbf{p}^\top A^\top A\, \mathbf{p}$ (minimise), $g(\mathbf{p}) = \mathbf{p}^\top \mathbf{p} - 1$:

$$\mathcal{L}(\mathbf{p}, \lambda) = \mathbf{p}^\top A^\top A\, \mathbf{p} - \lambda\,(\mathbf{p}^\top \mathbf{p} - 1)$$

Constrained least squares solution

Taking derivatives of $\mathcal{L}(\mathbf{p}, \lambda)$ with respect to $\mathbf{p}$: $2A^\top A\,\mathbf{p} - 2\lambda\,\mathbf{p} = \mathbf{0}$

$$A^\top A\,\mathbf{p} = \lambda\,\mathbf{p}$$ eigenvalue problem

The eigenvector $\mathbf{p}$ with the smallest eigenvalue λ of $A^\top A$ minimises the loss.

Rearrange the solution $\mathbf{p}$ to form the projection matrix P.

> **Question** Two gaps: why the **smallest** eigenvalue? And where does $\nabla_{\mathbf{p}}(\mathbf{p}^\top M\mathbf{p}) = 2M\mathbf{p}$ come from ($M = A^\top A$)?

Which eigenvector?

The Lagrangian gives only **candidates**: every unit eigenvector of $A^\top A$ satisfies $A^\top A\,\mathbf{p} = \lambda\,\mathbf{p}$ (12 of them).

Evaluate the loss at a candidate, using $\mathbf{p}^\top\mathbf{p} = 1$:

$$\|A\mathbf{p}\|^2 = \mathbf{p}^\top A^\top A\,\mathbf{p} = \mathbf{p}^\top(\lambda\,\mathbf{p}) = \lambda$$

The loss at each candidate is its eigenvalue → pick the eigenvector of the **smallest eigenvalue** of $A^\top A$.

In code: the last right singular vector of A, `np.linalg.svd(A)[2][-1]` — the same vector, without forming $A^\top A$ (numerically better).

The derivative of a quadratic form

$$q(p) = p^\top M p = \sum_i \sum_j M_{ij}\, p_i\, p_j$$

Differentiate with respect to p_k: the terms with $i = k$ give $\sum_j M_{kj} p_j$, the terms with $j = k$ give $\sum_i M_{ik} p_i$:

$$\frac{\partial q}{\partial p_k} = (Mp)_k + (M^\top p)_k$$

$$\nabla q = (M + M^\top)\, p = 2Mp \quad \text{for symmetric } M$$

Sanity check in 1D

$q(p) = mp^2,\ \ q'(p) = 2mp.$

Also useful

$\nabla_p (b^\top p) = b, \quad \nabla_p \|p\|^2 = 2p.$

So $\nabla_p \|Ap - b\|^2 = 2A^\top (Ap - b)$ — setting it to zero gives the normal equations $A^\top A p = A^\top b$ of ordinary least squares.

Extracting intrinsic and extrinsic parameters

We know that

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{M_{int}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{M_{ext}}$$

That is, the left 3×3 block:

$$\begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}}_{K} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}}_{R} = KR$$

> K is an upper triangular matrix and R is an orthonormal matrix: the product KR can be uniquely split ("decoupled") into K and R by the **RQ decomposition** (QR "from the other side"; unique with a positive diagonal of K).

Extracting intrinsic and extrinsic parameters (2)

The last column:

$$\begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} = K\,t$$

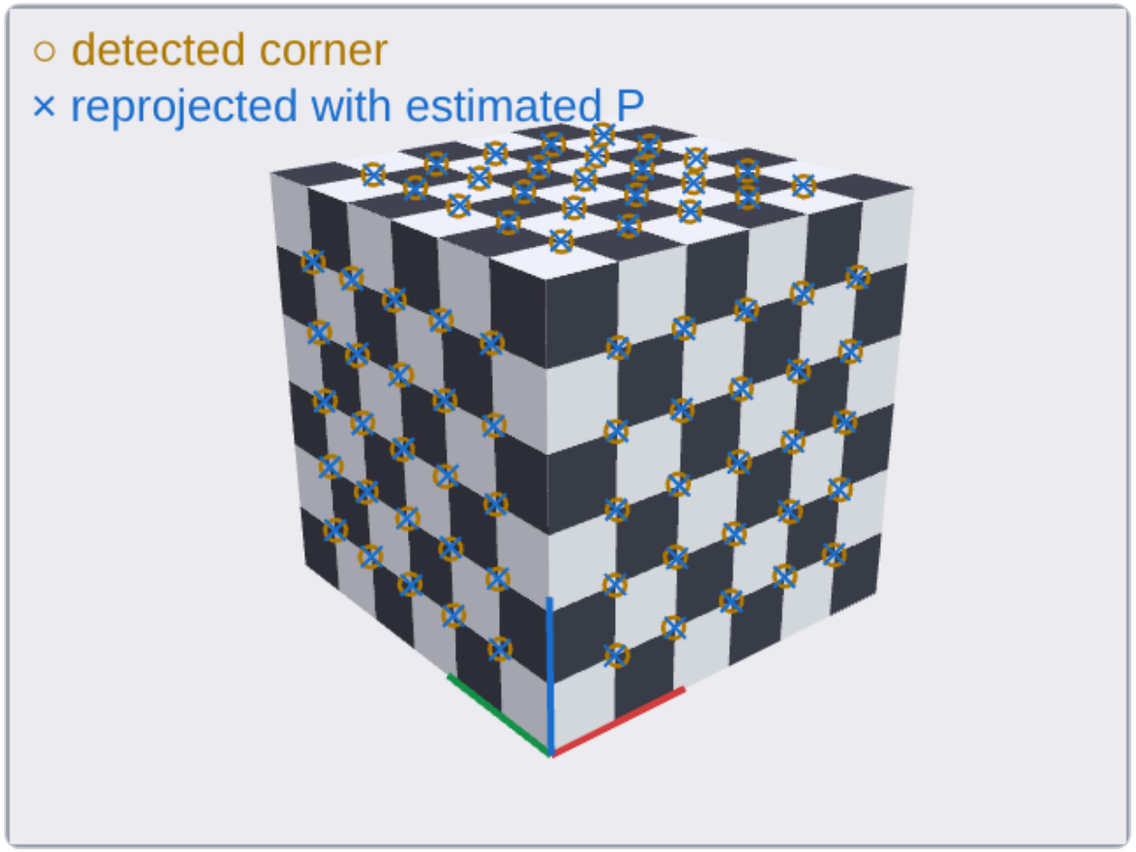
Therefore

$$t = K^{-1} \begin{bmatrix} p_{14} \\ p_{24} \\ p_{34} \end{bmatrix}$$

Details

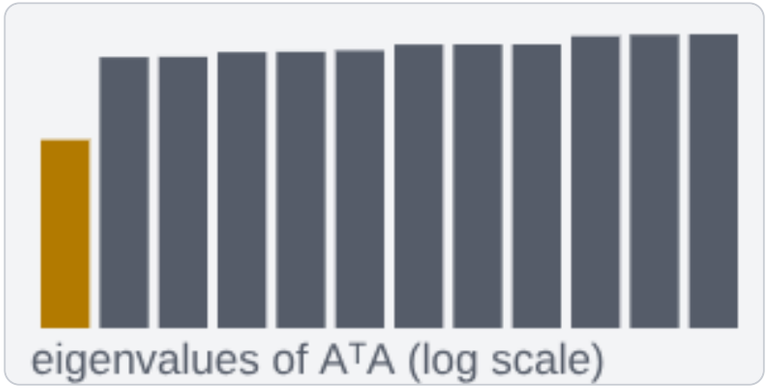
- P is known only up to scale (and sign): divide by k_{33}; if $\det$ of the left block is negative, flip the sign of P first, so that $\det R = +1$
- `scipy.linalg.rq`, `cv2.decomposeProjectionMatrix`
- camera centre directly: $\mathbf{c}_w = -R^\top t$

Calibration to play with



	true	estimate
f_x	800	788
f_y	800	789
o_x	330	330
o_y	235	237
skew s	0	0.2
camera pos. error	0	0.25

75 points · RMS reprojection error
0.73 px



DLT, and why one plane is not enough OPTIONAL

DLT = Direct Linear Transform

What we just did (steps 3–4):

- multiply out the denominators → equations **linear** in the entries of P
- stack them: $A\mathbf{p} = \mathbf{0}$
- solve **directly**: the eigenvector of $A^\top A$ with the smallest eigenvalue — no initial guess, no iterations

Used for many models: projection matrix, homography (lecture 3), triangulation.

All points on one plane: degenerate

Put the world frame on the board: every point has $z_i = 0$. The two rows of point i:

$$\begin{bmatrix} x_i & y_i & 0 & 1 & 0 & 0 & 0 & 0 & -u_i x_i & -u_i y_i & 0 & -u_i \\ 0 & 0 & 0 & 0 & x_i & y_i & 0 & 1 & -v_i x_i & -v_i y_i & 0 & -v_i \end{bmatrix}$$

- the columns of p_{13}, p_{23}, p_{33} (they multiply z_w) are **zero** in A: any values fit
- $A\mathbf{p} = \mathbf{0}$ has a whole subspace of solutions (several zero eigenvalues): P is not determined, however many points we take
- one view of a plane only gives the 3×3 map $(x, y, 1) \mapsto (\tilde{u}, \tilde{v}, \tilde{w})$: a **homography**, 8 DoF < 11

DLT calibration needs a 3D target (the cube). A flat checkerboard needs several views in different orientations (Zhang's method, next slide). Try "one face only" in the calibration widget.

Calibration in practice OPTIONAL

Zhang's method (what OpenCV does)

- a **flat** checkerboard, easy to print — but one view of a plane does not determine P (previous slide)
- take 10–20 photos in different orientations; each view gives a **homography** (plane → image, lecture 3)
- constraints from all views give K in closed form
- **non-linear refinement**: minimise the reprojection error over K, distortion and all board poses (Levenberg–Marquardt)

`cv2.findChessboardCorners` → `cv2.cornerSubPix` → `cv2.calibrateCamera`; ChArUco boards also work when partly occluded.

Algebraic vs geometric error

DLT minimises $\|Ap\|$, which has no physical meaning. What we care about is the distance in pixels between detected and reprojected points. Linear methods give the starting point; non-linear least squares finishes the job. Normalise the coordinates before DLT (Hartley): it is much more stable.

Good practice

- tilt the board; cover the whole image, especially the corners (distortion)
- fixed focus and zoom: refocusing changes K
- check the RMS reprojection error (well below 1 px)

Part 3

Perspective-n-Point (PnP)

Where is the object?

Given: a calibrated camera (K known), an object of known geometry (points X_i in the object frame), their pixels (u_i, v_i).

Find: the object pose R, t in the camera frame — 6 DoF.

> **Question** How many points do we need?

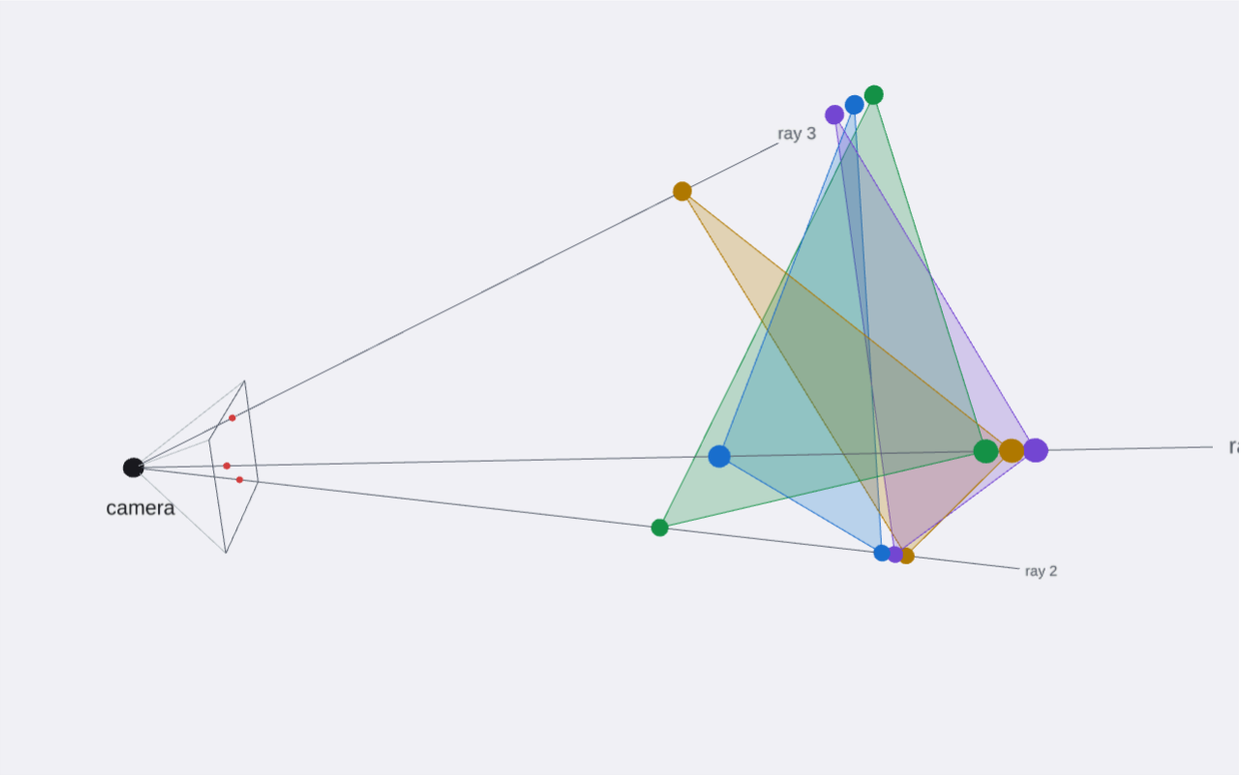
- each point: 2 equations → **3 points** (P3P)
- 3 points: up to **4 solutions**
- a 4th point usually picks the right one

Where it is used

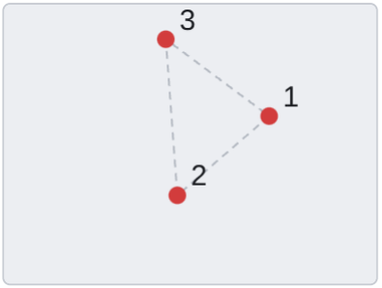
- fiducial markers (ArUco, AprilTag): 4 corners → pose of the marker
- object pose from a CAD model and detected keypoints
- localisation in a known map; augmented reality
- calibration: pose of the board in each view

Calibration: unknown K, 11 DoF, ≥ 6 points. PnP: known K, 6 DoF, ≥ 3 points.

Three points: up to four poses



the image



all four poses

side lengths: **1.49, 1.83, 1.49** m

distances along the rays: 2.94, 3.04, 2.19 m

every pose: the same triangle, the same three pixels

Three rays, one known triangle: up to **four** ways to place it with its corners on the rays. A 4th point picks the right one.

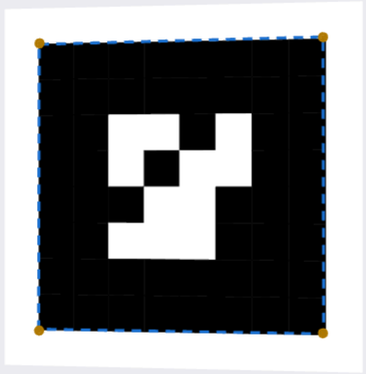
PnP as least squares

With many points and noise, minimise the **reprojection error**:

$$\min_{R,\,t} \sum_i \left\| \pi\big(K(RX_i + t)\big) - (u_i, v_i) \right\|^2, \qquad \pi(a,b,c) = (a/c,\ b/c)$$

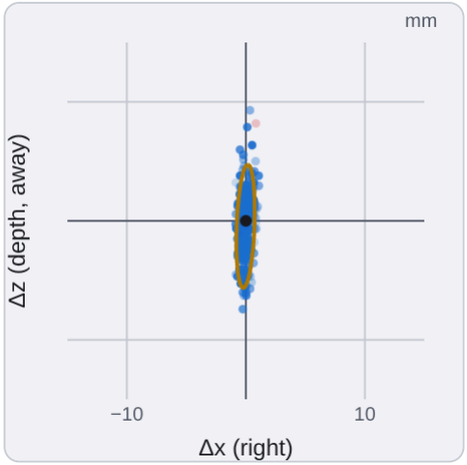
PnP to play with: how precise is the pose?

○ detected corners (radius 2σ)
-- marker at the estimated pose

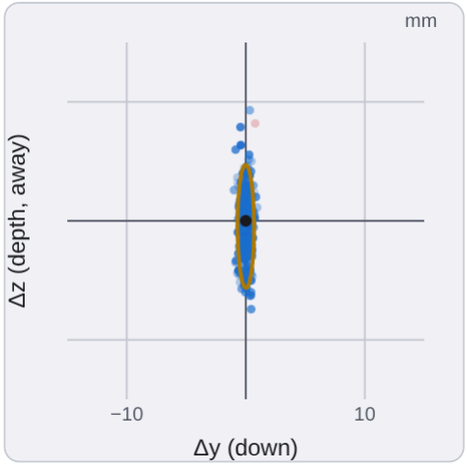


zoom ×1.5 · marker 160 px wide

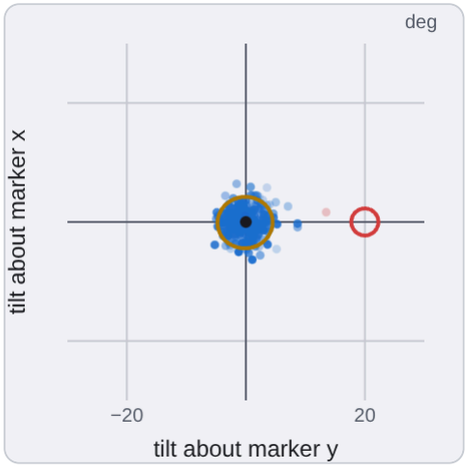
position error, top (mm)



position error, side (mm)



orientation error (deg)



σ over 300 solves

	pos. mm	rot. °
x	0.38	2.16
y	0.35	2.32
z	2.59	0.32

flipped: 0% (not in σ)

pos.: camera axes (z = depth)
rot.: marker axes (z = normal)

What to observe

1. **Depth is the weak direction**: the cloud is long along Δz. At 0.5 m: σ ≈ 0.4 mm sideways, ≈ 2.6 mm in depth.
2. **A frontal marker has a poorly known tilt**: distance 1 m, yaw 0° → tilt σ ≈ 9°, the dots form a ring around 0. Yaw 40° → ≈ 1.5°.
3. **Far away, the pose flips**: distance 2 m (marker ≈ 40 px) → 40–50% of the solves jump to the mirrored tilt (red, on the red ring).
4. **Rotation about the normal** is always precise (σ < 1.5°).

Remedies: larger or closer markers, view at an angle, more points, filtering over time.

Part 4

RANSAC

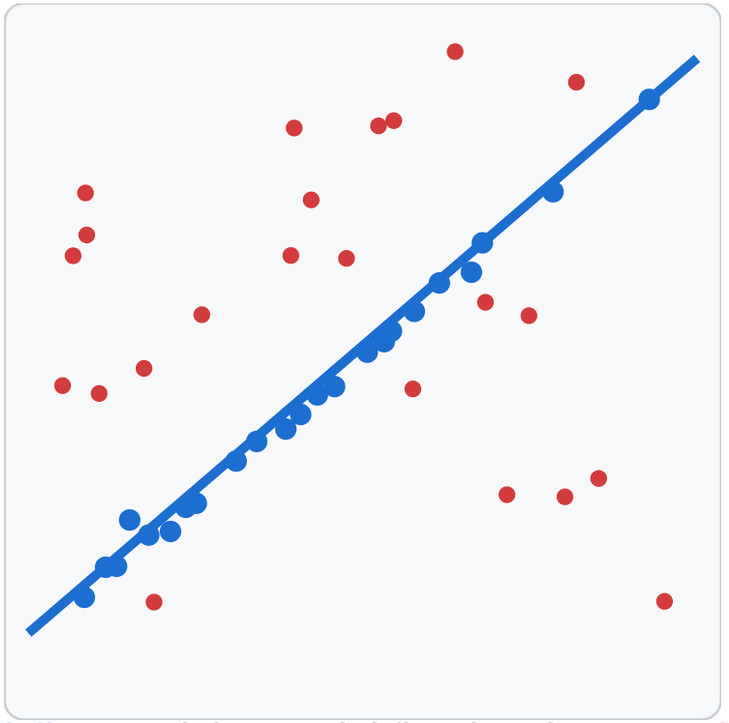
a way to deal with outliers

Outliers



measurements

→



inliers and the model fitted to them · outliers

- correspondences come from detectors and matchers; some are simply **wrong**: repeated texture, reflections, occlusions (10–50% for feature matches, lecture 3)
- least squares weights every point: one far outlier can move the solution arbitrarily far

RANdom SAmple Consensus

General RANSAC algorithm:

1. Randomly choose s samples. Typically s is the minimum number of samples to fit a model.
2. Fit the model to the randomly chosen samples.
3. Count the number M of data points (inliers) that fit the model within a measure of error ε.
4. Repeat steps 1–3 N times.
5. Choose the model that has the largest number M of inliers.

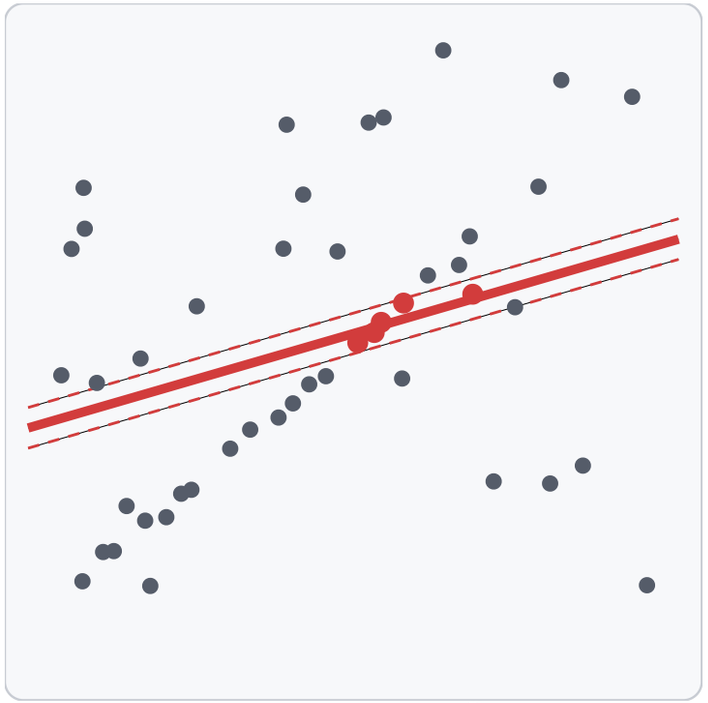
Finally: refit the model on all its inliers (least squares).

Minimal samples s

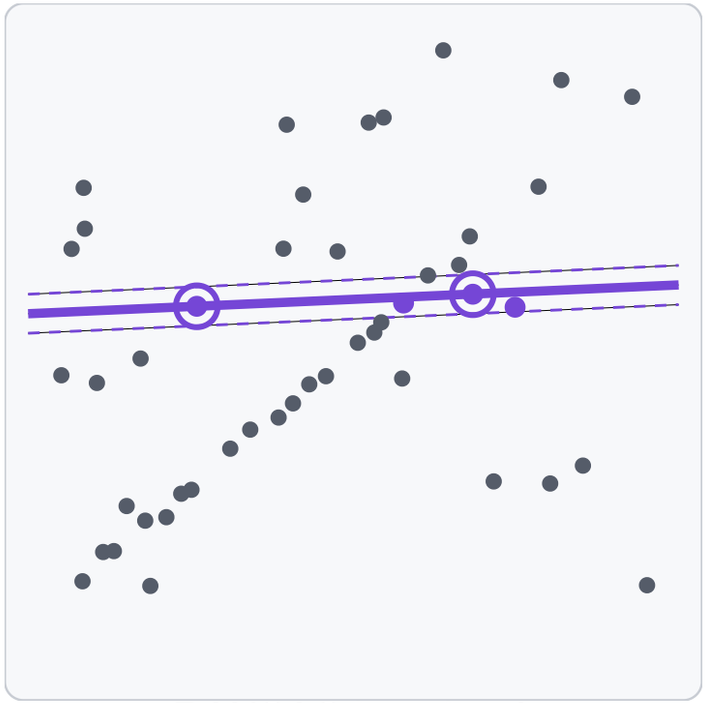
- line: 2 points
- PnP (P3P): 3 points
- calibration (DLT): 6 points
- homography: 4 points (lecture 3)

Fischler & Bolles, 1981.

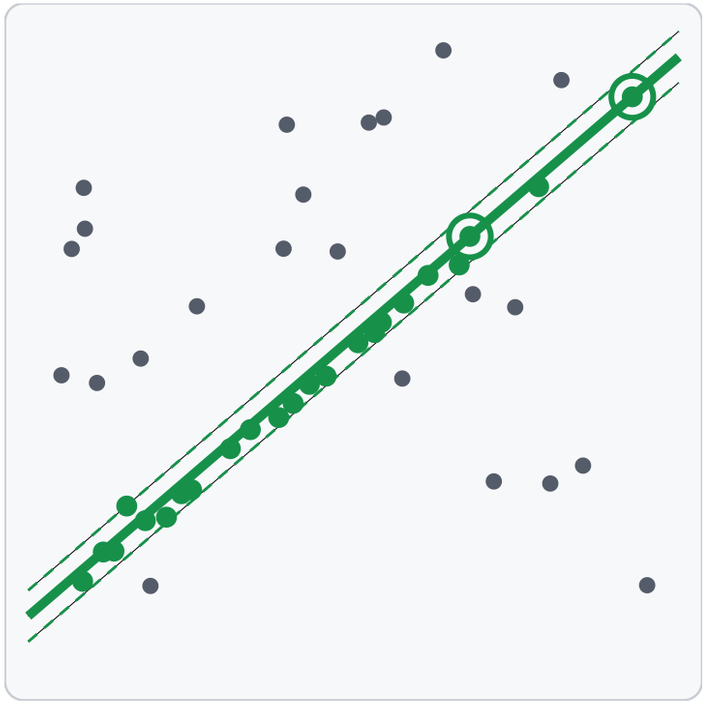
RANSAC example: line fitting



least squares fit
inliers: **5**



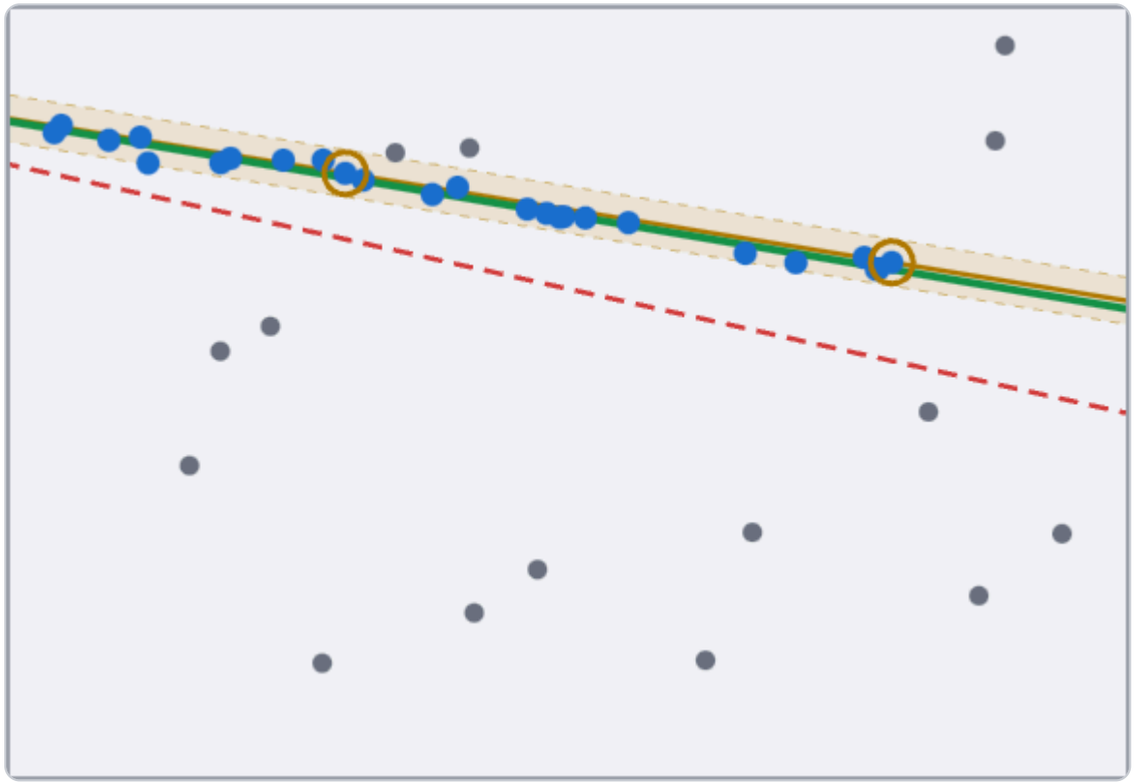
RANSAC iteration 1
inliers: **4**



RANSAC iteration i
inliers: **23**

Circled: the random sample ($s = 2$). Dashed: the band $\pm\varepsilon$; points inside it are inliers of that line.

RANSAC to play with: line fitting



iteration **11**

this sample: 25 inliers

best so far: 25 / 40

- current sample + band
- best / refit on inliers
- least squares, all points

$$N = \frac{\log(1-p)}{\log(1-w^s)}$$

$p = 0.99$, inlier fraction $w = 0.6$

model	s	w^s	N
line	2	0.36	**11**
P4P	4	0.13	**34**
DLT	6	0.047	**97**

How many iterations? OPTIONAL

w: fraction of inliers, s: sample size, p: desired success probability (e.g. 0.99).

- a sample is all inliers with probability w^s
- all N samples are contaminated: $(1 - w^s)^N$
- we want $(1 - w^s)^N \leq 1 - p$:

$$N \geq \frac{\log(1-p)}{\log(1-w^s)}$$

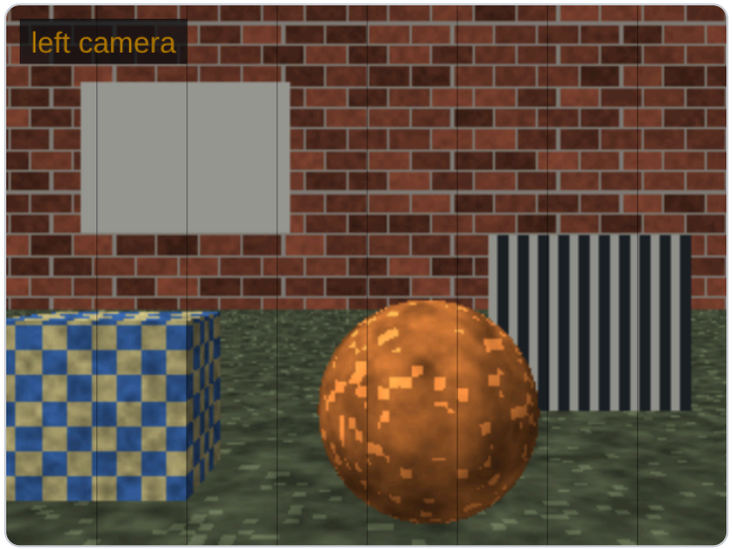
$p = 0.99$	$s = 2$	$s = 3$	$s = 4$	$s = 6$
$w = 0.9$	3	4	5	7
$w = 0.7$	7	11	17	37
$w = 0.5$	17	35	72	293
$w = 0.3$	49	169	567	6315

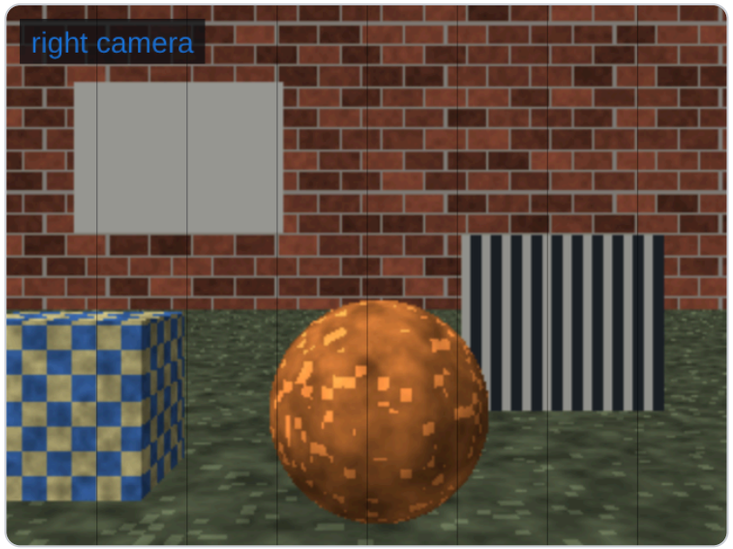
- does not depend on the number of points
- grows fast with s: use **minimal** solvers
- w unknown: update N from the best inlier ratio found so far (adaptive)
- threshold $\varepsilon \approx 2$–3σ of the measurement noise

Part 5

Simple stereo

Two cameras, two views





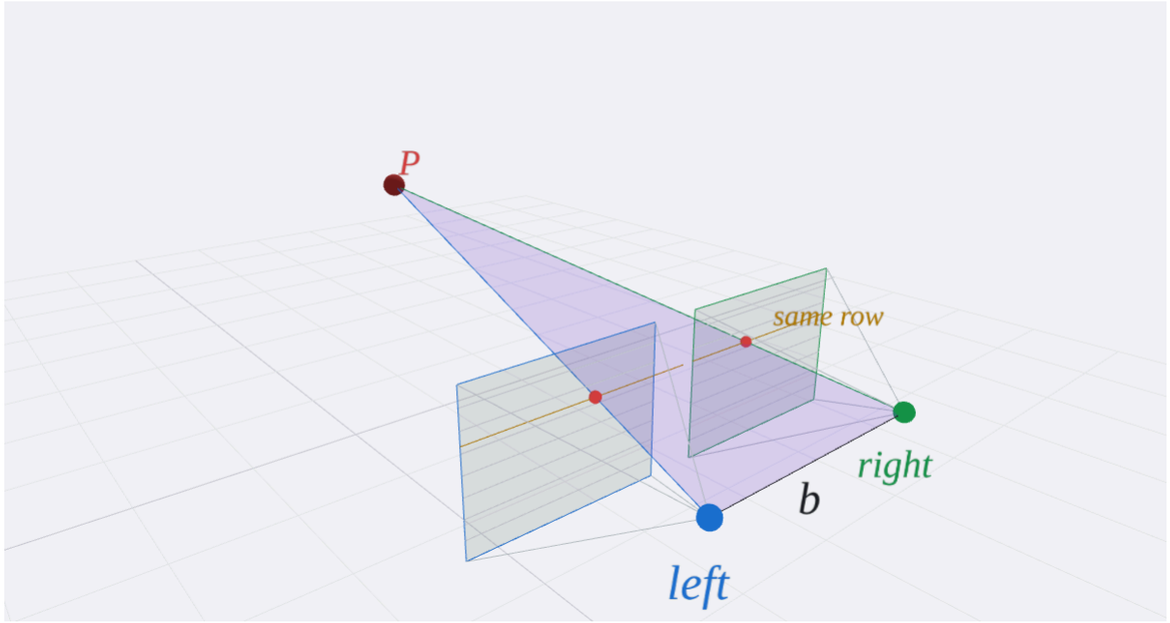
The same scene from two cameras side by side (the right one is 20 cm to the right).

Switch between the views:

- **close** objects (the sphere, the box) jump a lot
- **far** objects (the back wall) barely move

The shift (**disparity**) tells us the depth: the closer the object, the larger the shift.

Rectified stereo: the same row in both images





$v_l = v_r = 160.5$

$u_l = 422.3,\ u_r = 240.5$

$d = u_l - u_r = 181.8$ px

Definition

A stereo pair is **rectified** if its images look as if taken by two **identical** cameras (same K) with the **same orientation**, the right one shifted only along the x axis of the left one, by the **baseline** b.

Disparity and depth

Point (x, y, z) in the left camera frame; the right camera sits at $(b, 0, 0)$:

$$u_l = f\frac{x}{z} + o_x, \qquad u_r = f\frac{x-b}{z} + o_x$$

Same row in both images: $v_l = v_r = v$.

The **disparity**:

$$d = u_l - u_r = \frac{f\,b}{z} \quad \Longrightarrow \quad z = \frac{f\,b}{d}$$

$$x = \frac{b\,(u_l - o_x)}{d}, \qquad y = \frac{b\,(v - o_y)}{d}$$

- depth is **inversely** proportional to disparity
- disparity is proportional to the baseline b
- far away: $d \to 0$; at infinity $d = 0$ (the same pixel in both images)
- near: large d — very close points may be visible in one camera only

Rectified stereo

- the plane through P and both centres contains the baseline, so it cuts both image planes along the **same row**
- the match of a pixel is searched along **one row** only
- the two rays meet at P: **triangulation** — the formulas on the left

Rectification: warping the two images of a general pair (one homography each) into this form, `cv2.stereoRectify`. General case: lecture 3.

Depth precision degrades with z^2

Disparity is measured to a fraction of a pixel. How does an error Δd affect depth?

$$\left|\frac{\partial z}{\partial d}\right| = \frac{fb}{d^2} = \frac{z^2}{fb} \quad \Longrightarrow \quad \Delta z \approx \frac{z^2}{f\,b}\,\Delta d$$

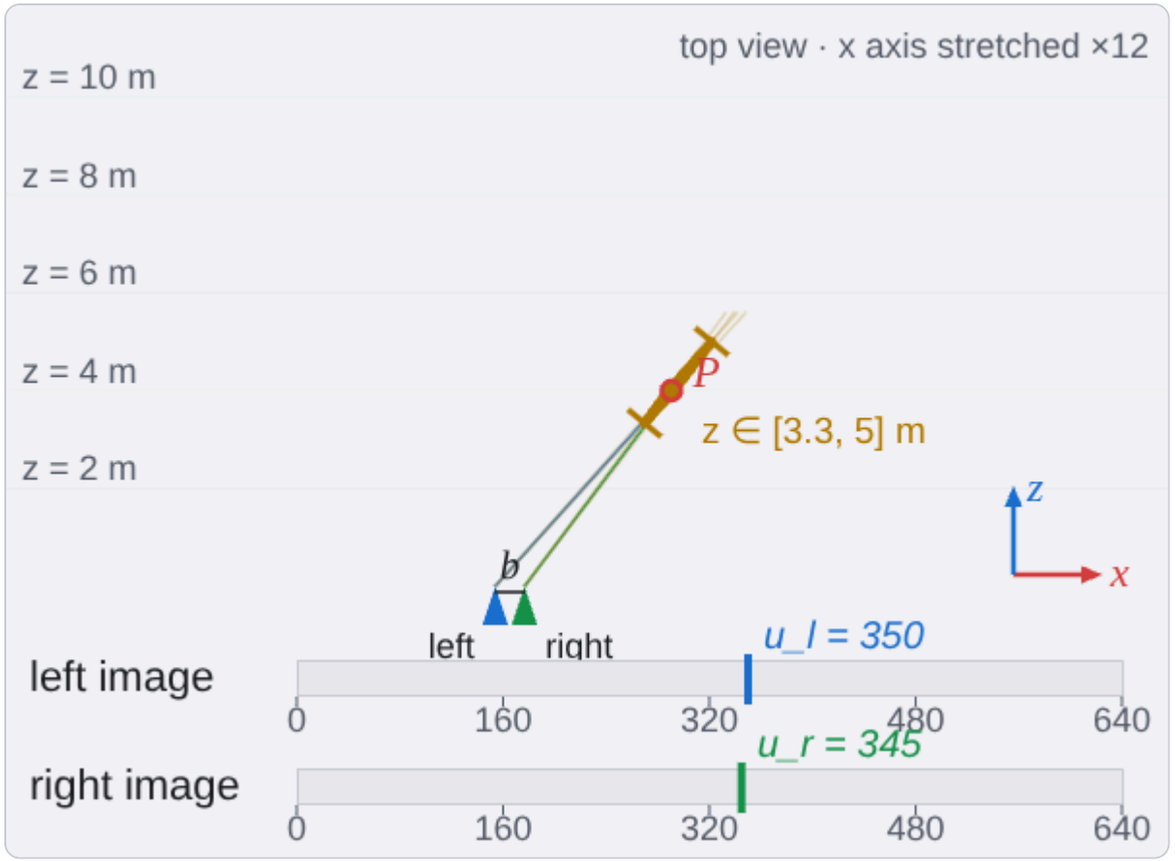
Example: $f = 600$ px, $b = 10$ cm, $\Delta d = 0.25$ px:

z	1 m	5 m	10 m
Δz	4 mm	10 cm	42 cm

For a robot

- stereo is precise for grasping at arm's length, coarse for distant obstacles
- more precision at range: larger baseline or focal length (narrower field of view)
- far away the two rays are almost parallel: the region of 3D positions consistent with both pixels (± the matching error) stretches along the ray (next slide)

Depth uncertainty to play with



$$u_l = f\,x/z + o_x = 350$$

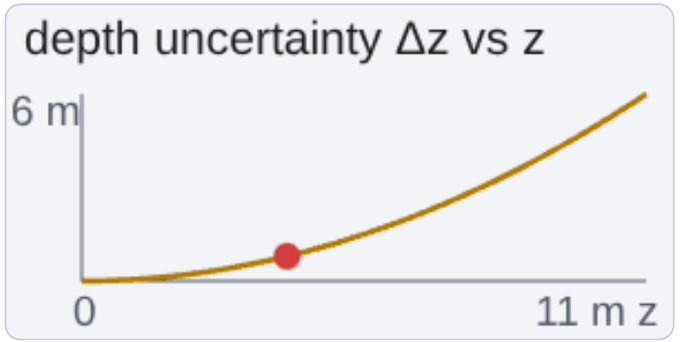
$$u_r = f\,(x-b)/z + o_x = 345$$

$$d = u_l - u_r = 5 \text{ px}$$

$$z = \frac{b\,f}{d} = \frac{0.05 \cdot 400}{5} = 4 \text{ m}$$

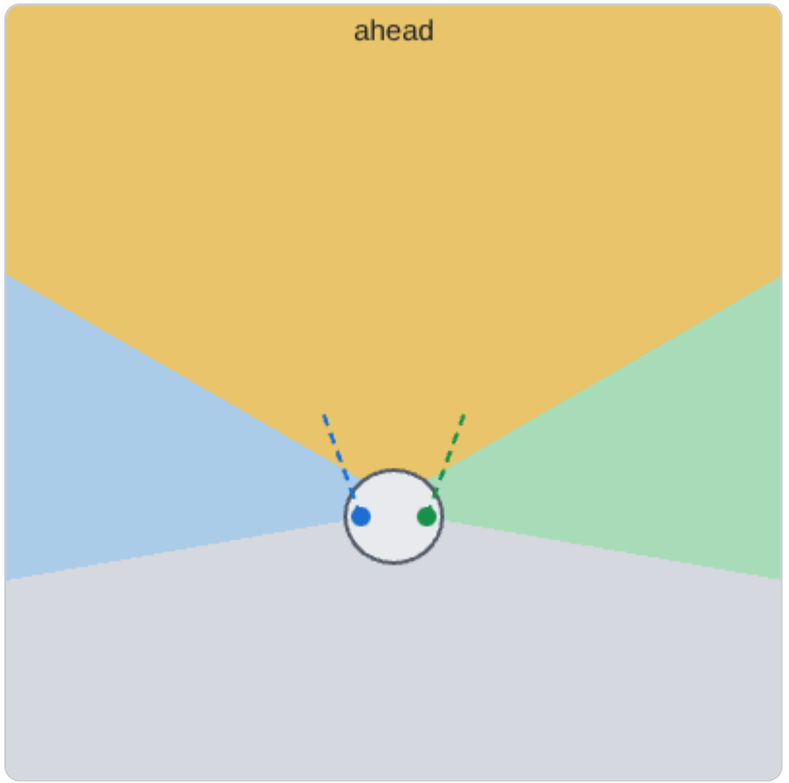
$$\Delta z \approx \frac{z^2}{bf}\,\Delta d = 0.8 \text{ m}$$

$\Delta d = 2e = 1$ px (each pixel ± 0.5)



Drag P.

Hunters and prey: where to put the eyes



human

total field ≈ **200°**

binocular (stereo) ≈ **120°**

blind ≈ 160°

- both eyes: stereo
- left eye only
- right eye only
- blind



hunter: tawny owl, eyes forward

Charlie Marshall, CC BY 2.0, Wikimedia Commons



prey: rabbit, eyes on the sides

caroline legg, CC BY 2.0, Wikimedia Commons

Prey: eyes on the sides, almost 360° coverage, little stereo. Hunters: eyes forward, a wide binocular field for depth. Robots often do both: surround cameras for awareness, a forward stereo pair for precise depth.

Finding correspondences: block matching

- rectified: the match of pixel (u, v) lies on the **same row** v, at $u - d$ with $0 \leq d \leq d_{\max}$
- compare a small window around (u, v) in the left image with windows along the row in the right image
- take the d with the lowest cost ("winner takes all")

Window costs

over the pixels of the two windows L (left) and R (right):

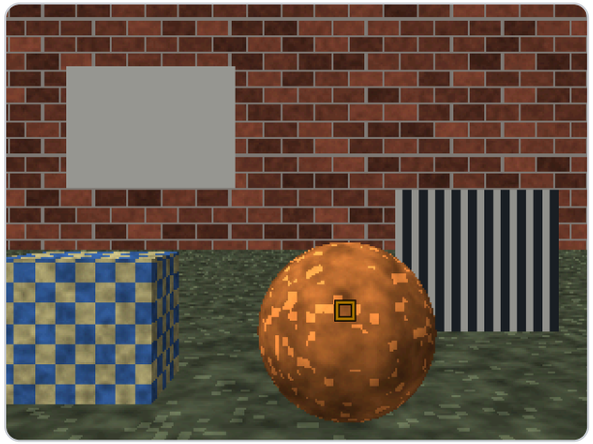
- **SAD**, sum of absolute differences: $\sum |L - R|$
- **SSD**, sum of squared differences: $\sum (L - R)^2$

NCC, normalised cross-correlation: robust to brightness differences between the cameras.

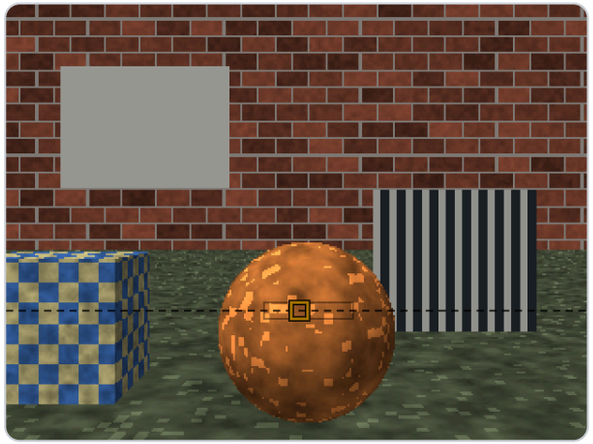
> A 2D search becomes a 1D search: this is why we rectify.

Block matching to play with

left image



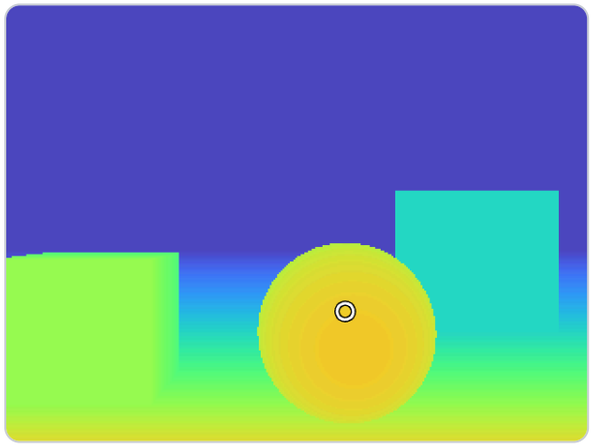
right image



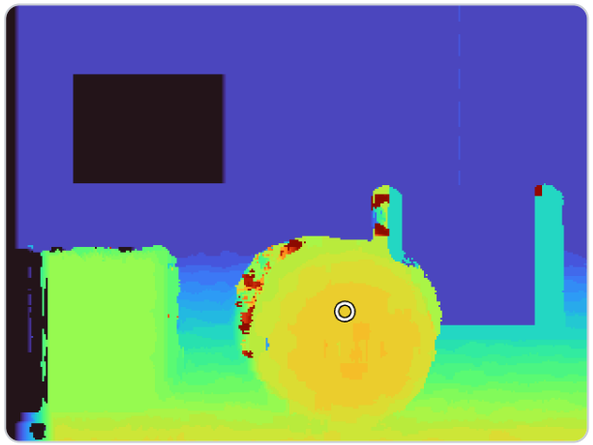
window 9×9



true disparity



block matching



wrong by > 1 px: **18.6%**



When matching fails

- **no texture** (white wall): every window looks the same
- **repetitive texture** (stripes, tiles): several equally good matches
- **occlusions**: points seen by only one camera have no match
- **specular / transparent** surfaces: appearance differs between the views
- **window size**: small → noisy; large → blurred depth edges, and the window assumes a fronto-parallel surface (foreshortening)

Beyond block matching

- **semi-global matching** (SGM): adds smoothness along several directions — `cv2.StereoSGBM`
- **active stereo**: an infrared projector paints a random dot pattern, giving texture to white walls (e.g. Intel RealSense D400)
- **learned stereo**: neural networks trained on large (often synthetic) datasets, e.g. RAFT-Stereo
- other depth sensors: structured light, time-of-flight, lidar

Summary

- $P = K[R \mid t]$, 3×4, 11 DoF; $t = -RC$
- calibration (DLT): 2 linear equations per point, $\min \|Ap\|$ s.t. $\|p\| = 1 \rightarrow$ eigenvector of the **smallest** eigenvalue of $A^\top A$; RQ $\rightarrow K, R$
- in practice: planar board, Zhang's method, refine the reprojection error
- PnP: known K, 6 DoF, P3P gives ≤ 4 solutions; refine by least squares
- RANSAC: minimal samples + consensus; $N = \log(1-p)/\log(1-w^s)$
- rectified stereo: $z = fb/d$, $\Delta z \approx z^2 \Delta d/(fb)$
- correspondence along the row: block matching; fails without texture, with repetitions and occlusions

> Next lecture: homographies and image stitching, features, two-view geometry for general camera pairs.

Exercises

1. K: $f_x = f_y = 500$, $(o_x, o_y) = (320, 240)$; $R = I$, $t = (0, 0, 2)$. Project $X_w = (1, 0.5, 3)$. Where is the camera centre?
2. Show that the camera centre C satisfies $P\,(C, 1)^\top = 0$.
3. Why does DLT need at least 6 points? Show that points on one plane $z_w = 0$ leave the third column of P undetermined.
4. Show that $\nabla_p \left(p^\top M p\right) = \left(M + M^\top\right) p$.
5. RANSAC for a homography ($s = 4$) with 30% outliers: how many iterations for $p = 0.99$?
6. Stereo with $b = 12$ cm, $f = 700$ px: a point has disparity 21 px. Its depth? The depth error for $\Delta d = 0.5$ px?

Self-study: taking good pictures OPTIONAL

Practical aspects that matter for robot cameras:

- lens vs pinhole, depth of field (lecture 1)
- aperture, gain (ISO), exposure time — the exposure triangle
- motion blur; **rolling** vs **global** shutter (fast motion!)
- lens distortion (lecture 1, part 6)

Materials:

- First Principles of Computer Vision: Lenses; Depth of field; Lens related issues
- B. Ciechanowski, Cameras and Lenses (interactive; skip "Waves", continue at "Manipulating rays")

Feedback



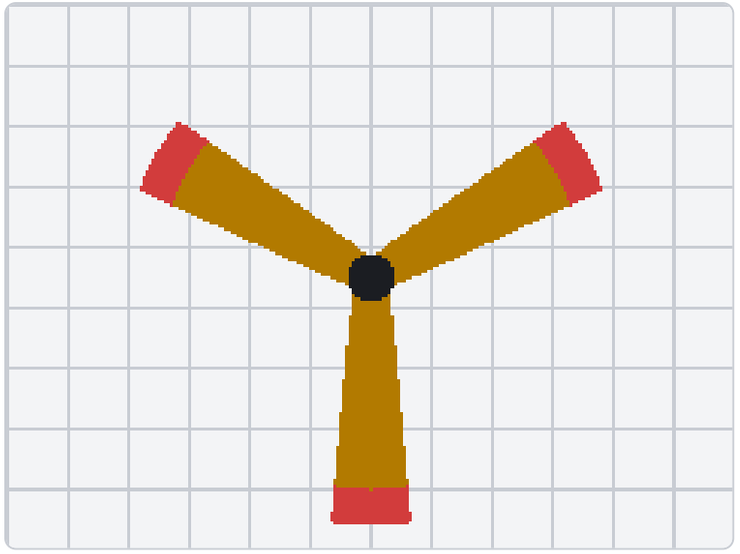
Please spend a minute: your feedback helps me improve the next lecture.

`tinyurl.com/mimuw-rc-feedback`

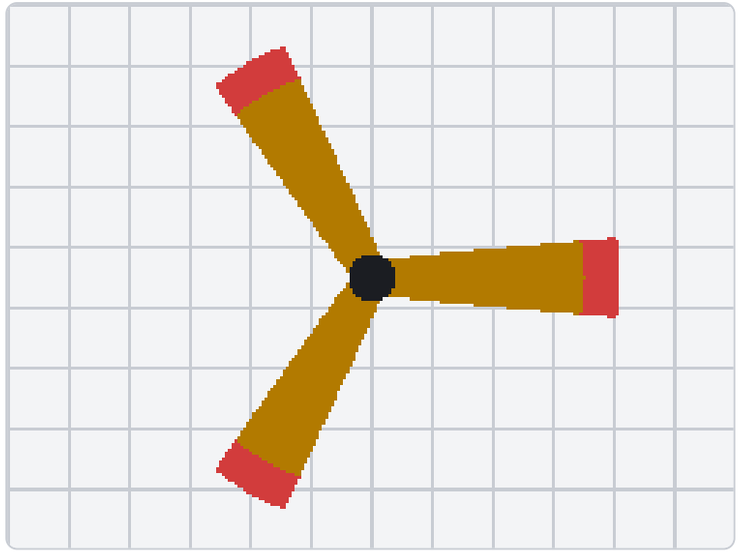
Anonymous, no login, three questions.

Appendix: rolling vs global shutter APPENDIX

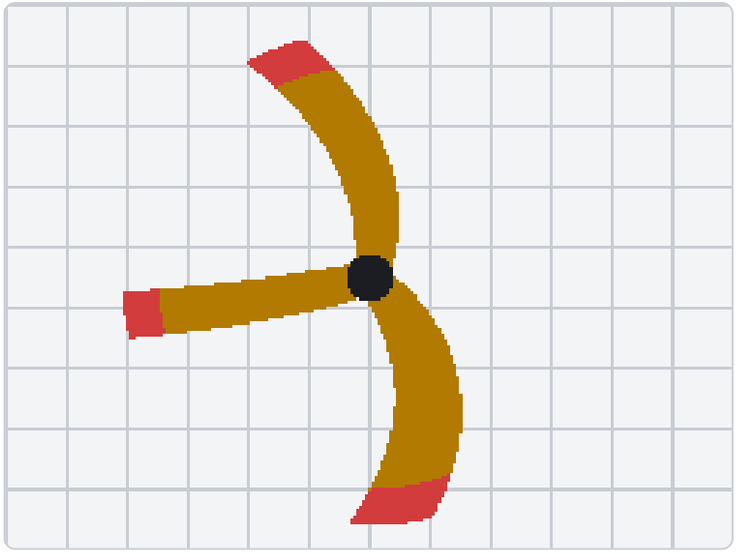
the scene (slowed down)



global shutter: all rows at once



rolling shutter: row by row



Global shutter: all pixels are exposed during the same interval, then read out. The image is one moment in time, as the pinhole model assumes.

Rolling shutter (most CMOS sensors: phones, webcams): rows are exposed and read one after another, top to bottom. Each row sees a different moment: rotating objects bend, moving objects skew.

For a robot: a moving camera skews the whole scene, which breaks calibration, PnP and visual odometry. Use a global-shutter camera, or model the time of each row.